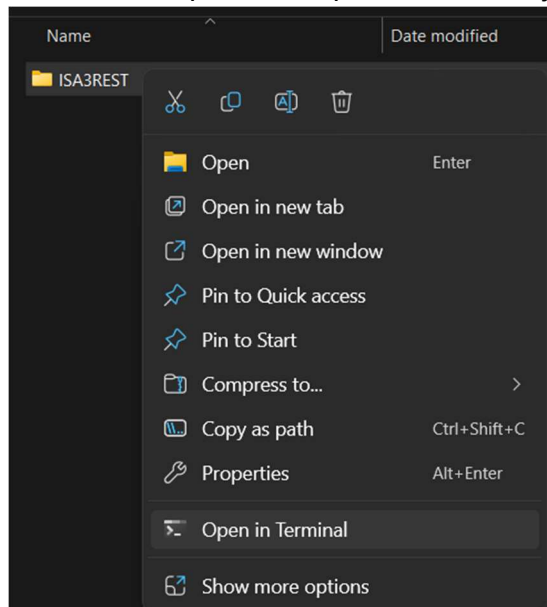


Exercises REST API loomise juhend

1. Avame VSCode'is kausta, kus soovime projekti luua, selleks on kaks varianti:

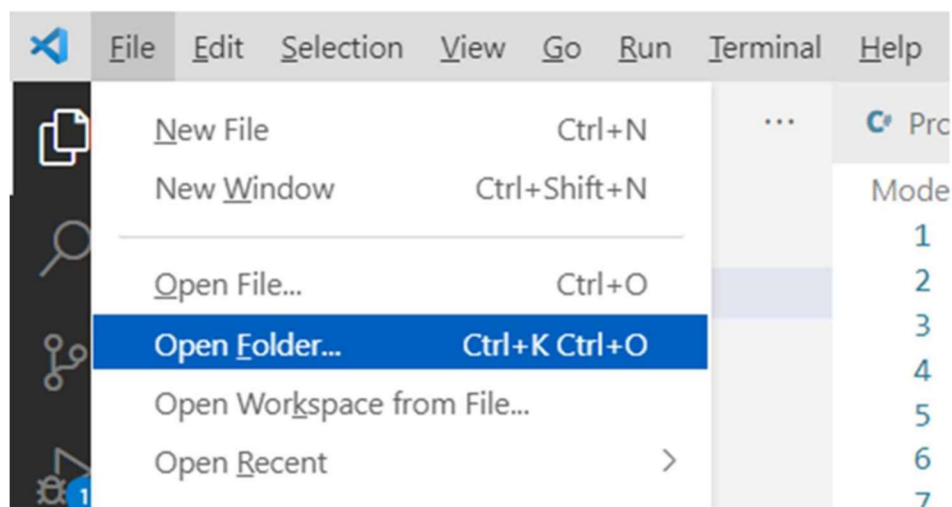
i. Valida File Explorer'is "Open in terminal" ja kirjutada käsureal "code ."



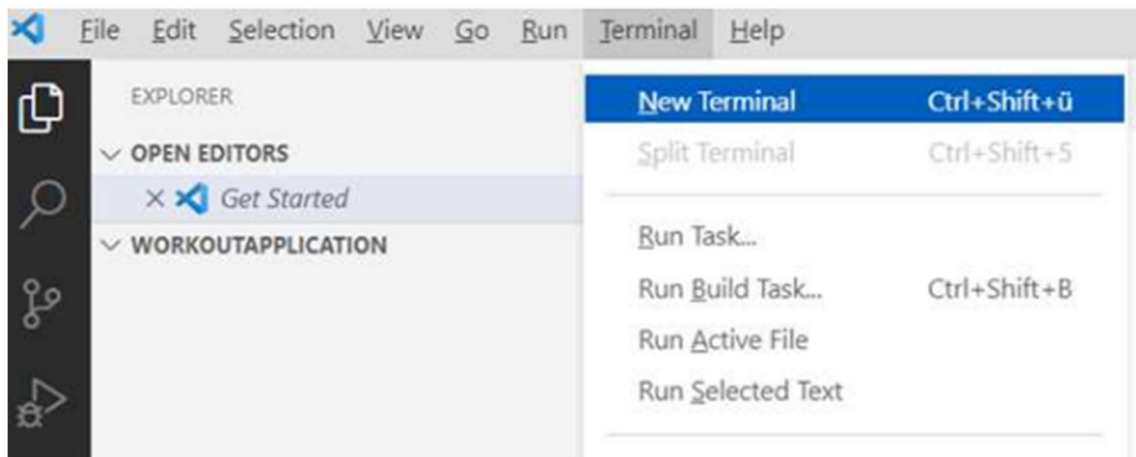
Select Käsuviip

```
C:\kool\hajusrakendused>mkdir WorkoutApplication  
C:\kool\hajusrakendused>cd WorkoutApplication  
C:\kool\hajusrakendused\WorkoutApplication> code .  
C:\kool\hajusrakendused\WorkoutApplication>
```

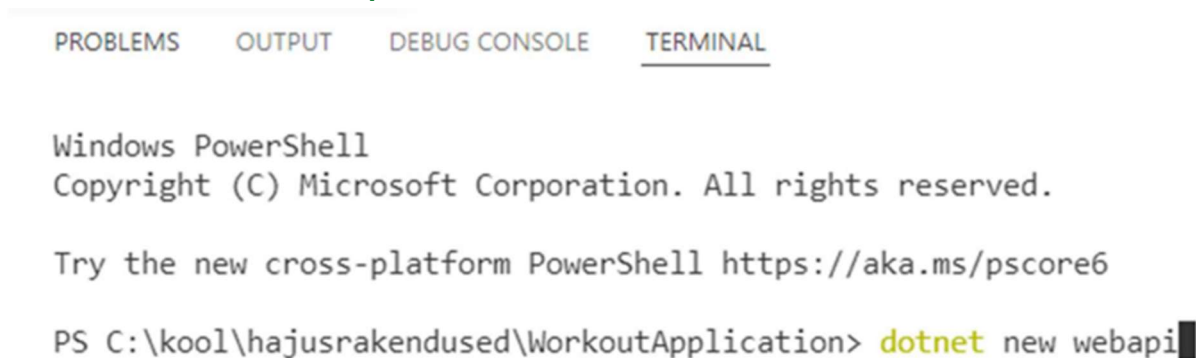
ii. Läbi VSCode'i



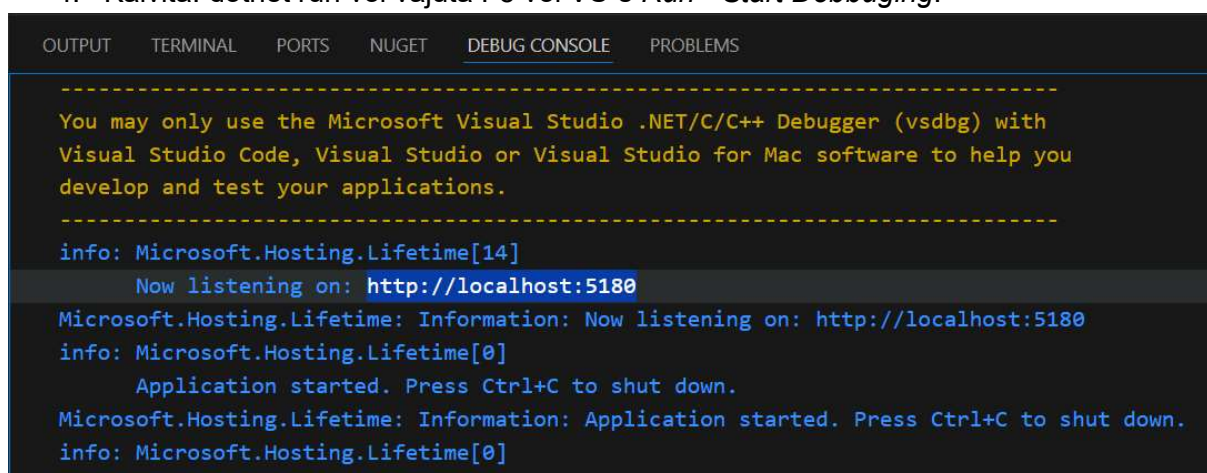
2. VSCode'is on sisseehitatud võimalus avada käsurida(nii powershell kui ka CMD)



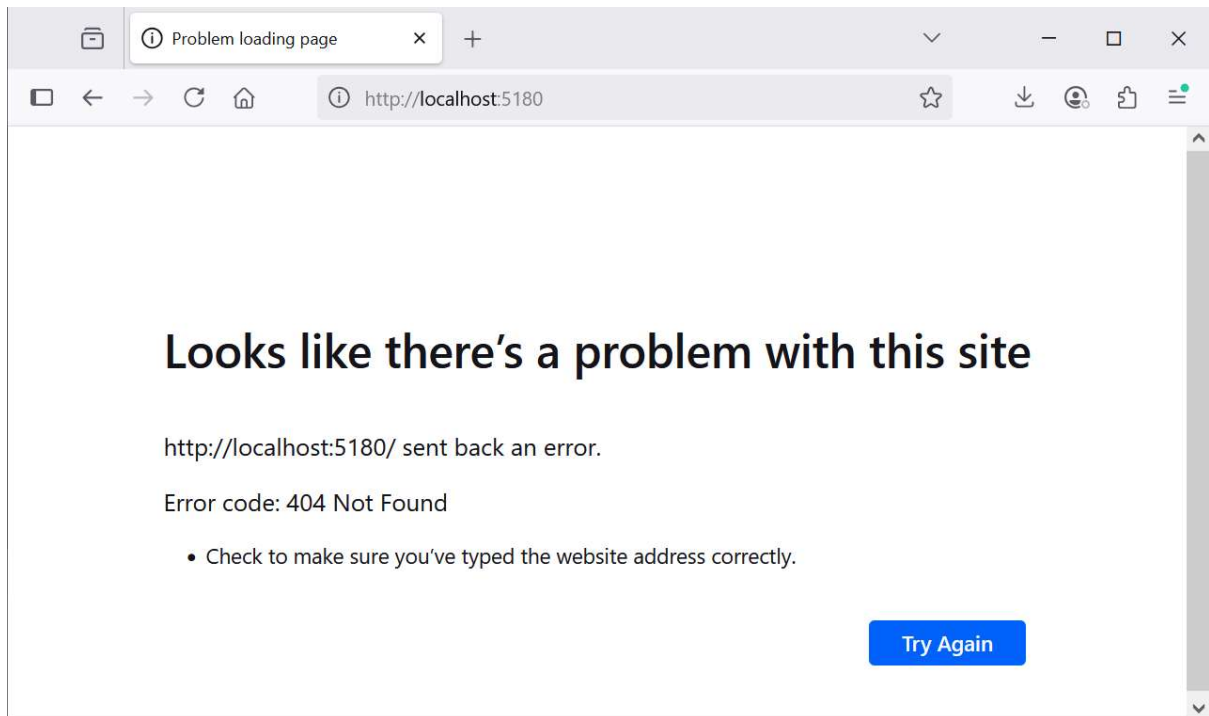
3. Loo projekt käsura käsuga. Teeme ASP.NET Core WEB API rakenduse:
dotnet new webapi



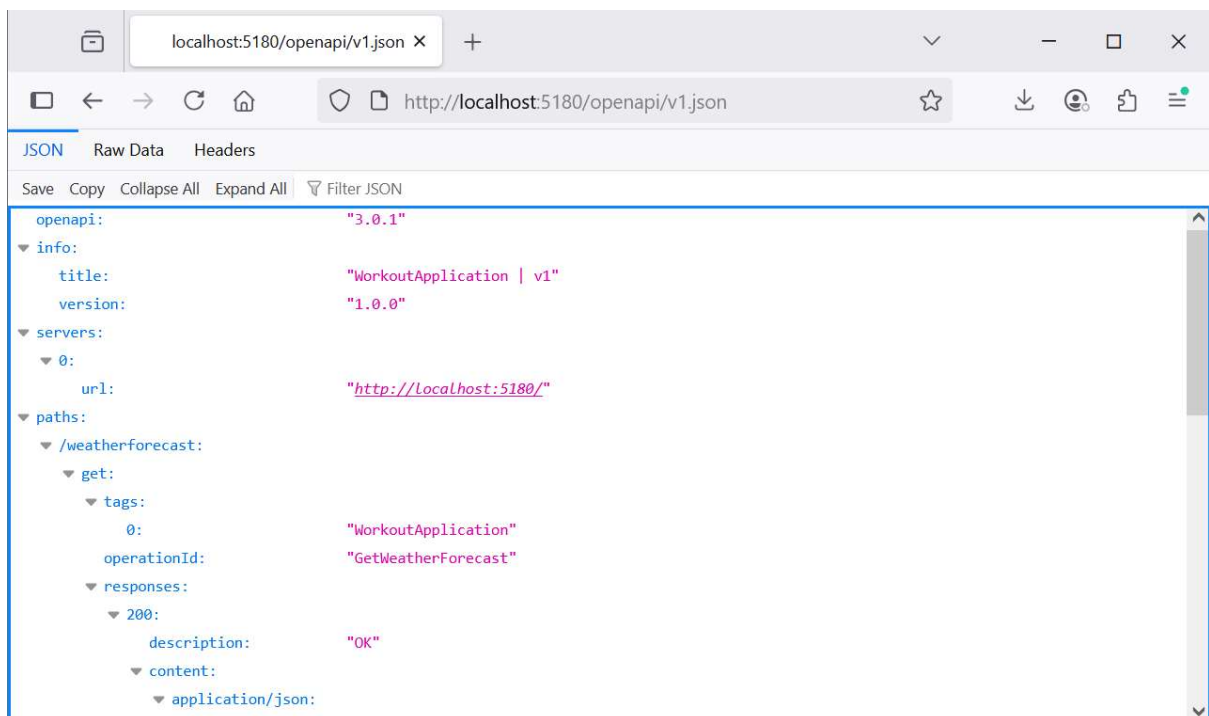
4. Käivita: dotnet run või vajuta F5 või VS-s *Run->Start Debugging*.



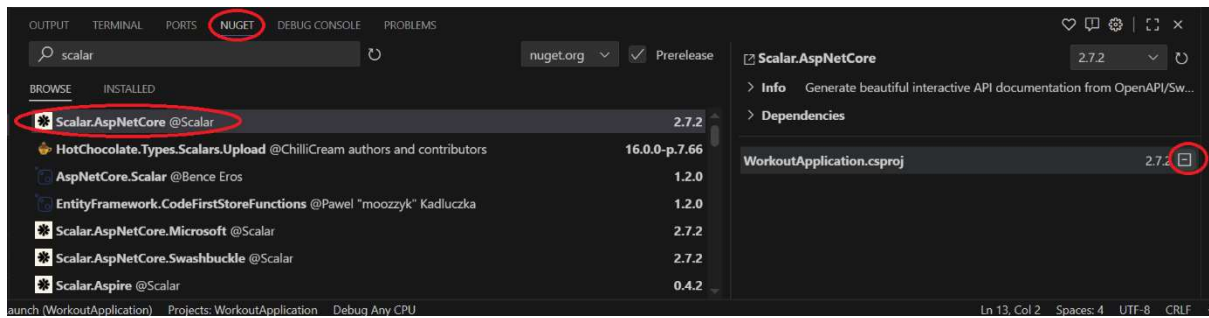
5. Kui veebilehitsejaga avad selle aadressi (port võib erineda), siis see pole kättesaadav.



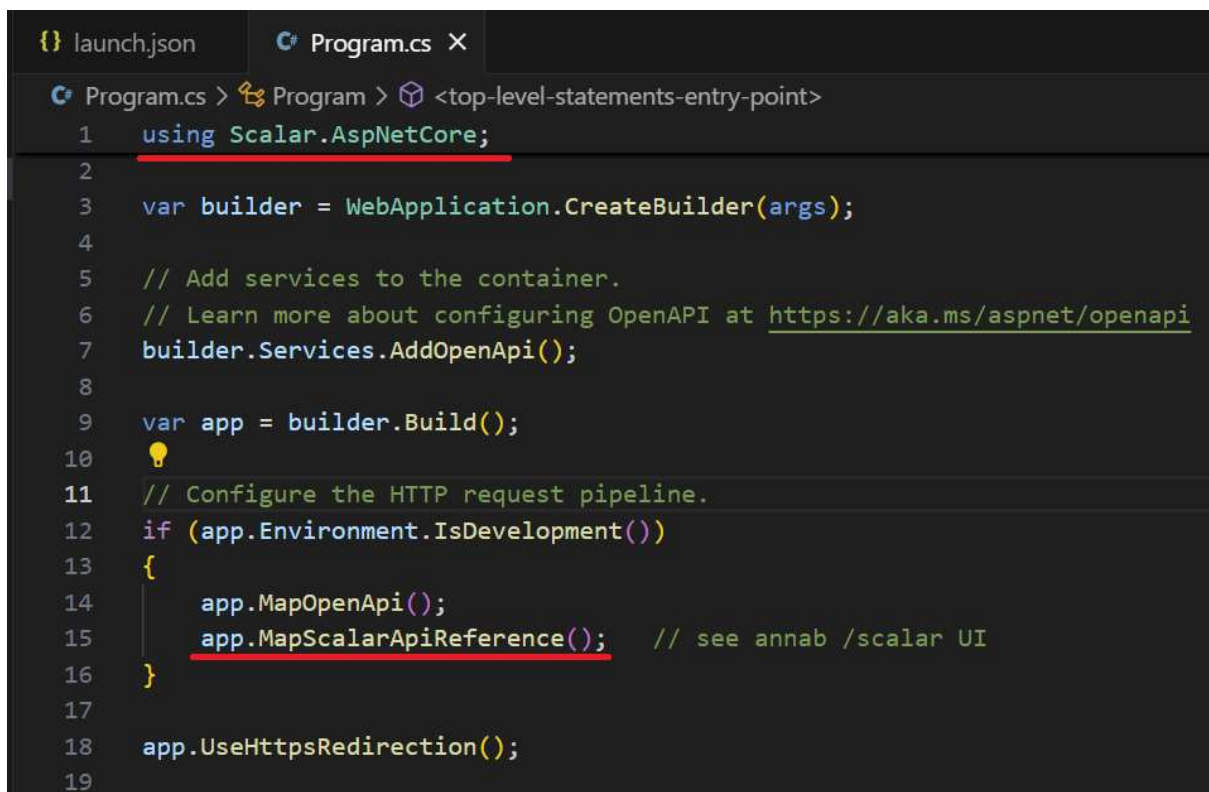
6. Kui lisad aadressile `/openapi/v1.json`, siis peaksid saama järgneva sisu:



7. Lisa projektile `Scalar.AspNetCore` pakett. Seda saad teha näite VS Code-s nagu järgneval pildil:



8. Lisa programmi koodi kaks rida nagu allpool näidatud. Nende abil lisatakse kasutajaliides, mis oskab OpenApi dokumentatsiooni esitleda veebilehitsejas.



9. Lisame .NET arenduse HTTPS sertifikaadi Windows sertifikaatide hulka: **dotnet dev-certs https --trust**

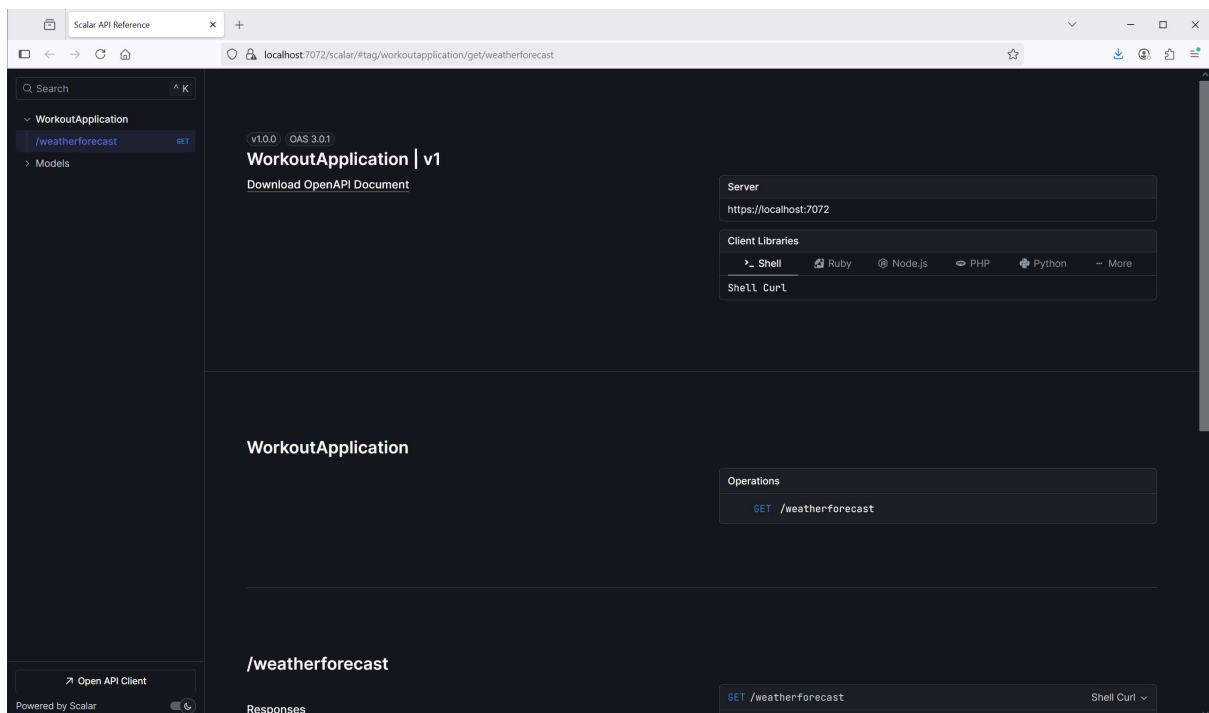
- i. Firefox'i sertifikaadid on Windowsist eraldiseisvad. Firefox'i sertifikaadi usaldamiseks saab lisaks eelnevale avada lehe about:config ja määrata security.enterprise_roots.enabled väärtuseks true. Veel infot : <https://docs.microsoft.com/en-us/aspnet/core/security/enforcingssl?view=aspnetcore-6.0&tabs=visual-studio#trust-the-aspnet-core-httpsdevelopment-certificate-on-windows-and-macos>

PS C:\kool\hajusrakendused\WorkoutApplication> dotnet dev-certs https --trust

10. Projekti juurkaustas (kus on Program.cs, csproj jne) tee uus kaust (kui veel pole) nimega **.vscode** ja selle sisse tee fail **launch.json** järgneva sisuga:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": ".NET Launch",
      "type": "coreclr",
      "request": "launch",
      "program":
"${workspaceFolder}/bin/Debug/net9.0/ExercisesApi.dll",
      "cwd": "${workspaceFolder}",
      "launchSettingsProfile": "https",
      "env": { "ASPNETCORE_ENVIRONMENT": "Development" },
      "serverReadyAction": {
        "action": "openExternally",
        "pattern": "\\bNow listening on:\\s+(https?://\\S+)",
        "uriFormat": "%s/openapi/scalar"
      }
    }
  ]
}
```

11. Kui nüüd VS Code-s uuesti programmi käivitad (*Start Debugging*), siis peaks avanema automaatselt API dokumentatsioon veebilehitsejas.



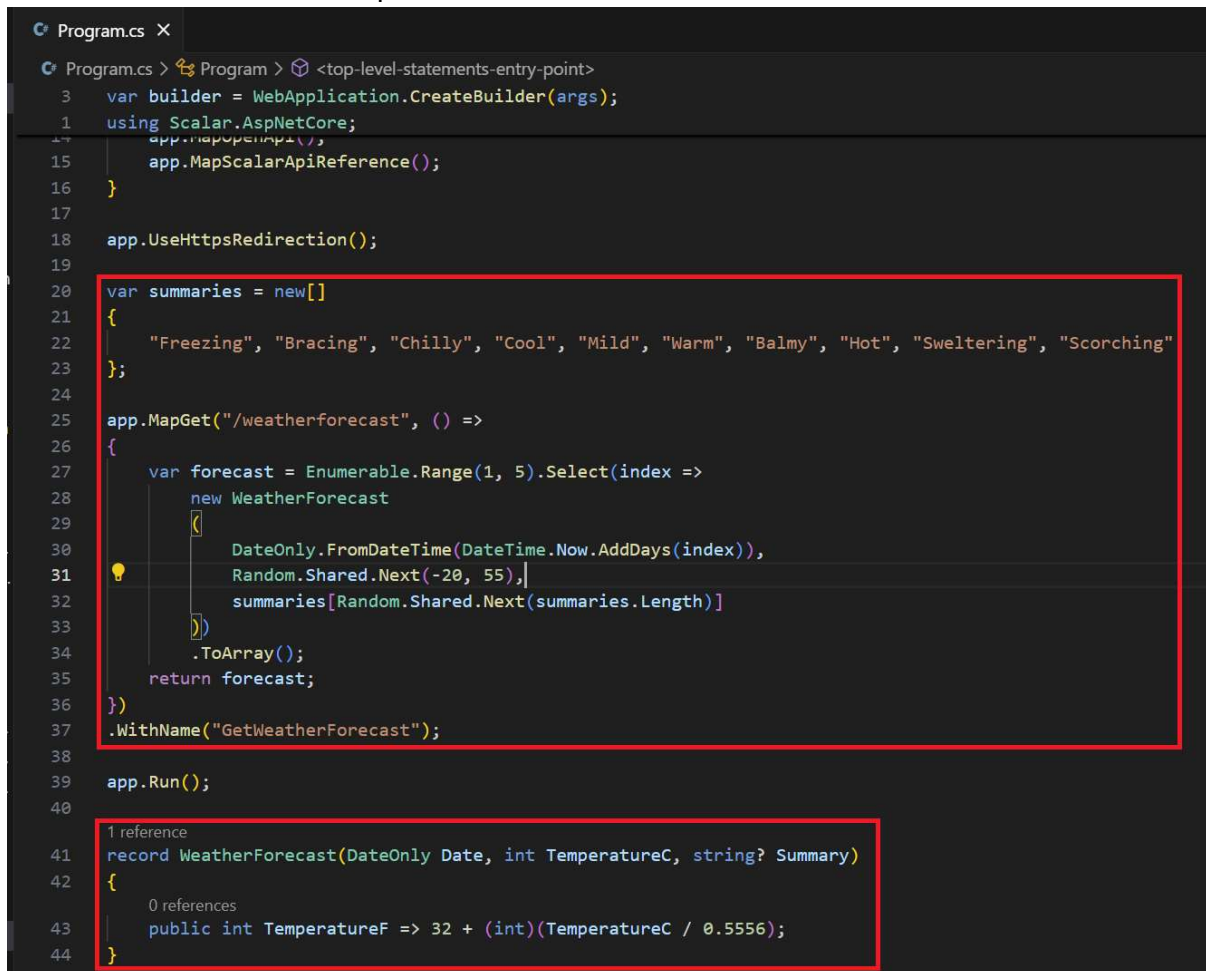
12. Kui saad 404 veateate, siis võid proovida käsurealt järgmisi käske:

dotnet clean

dotnet restore

dotnet build

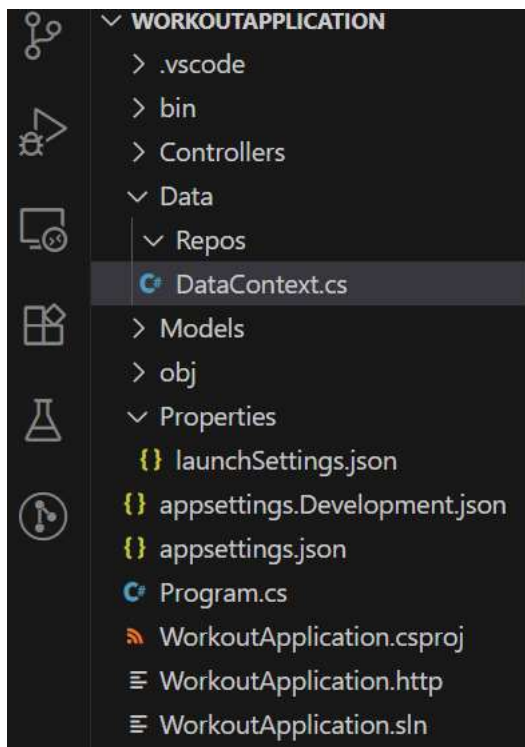
13. Kustutame järgmised automaatselt genereeritud koodi tükid, kuna me ei taha näha /weatherforecast endpoint-i meie rakenduses



```
Program.cs X
Program.cs > Program > <top-level-statements-entry-point>
3  var builder = WebApplication.CreateBuilder(args);
1  using Scalar.AspNetCore;
   app.MapOpenApi();
15  app.MapScalarApiReference();
16  }
17
18  app.UseHttpsRedirection();
19
20  var summaries = new[]
21  {
22      "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
23  };
24
25  app.MapGet("/weatherforecast", () =>
26  {
27      var forecast = Enumerable.Range(1, 5).Select(index =>
28          new WeatherForecast
29          {
30              DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
31              Random.Shared.Next(-20, 55),
32              summaries[Random.Shared.Next(summaries.Length)]
33          })
34          .ToArray();
35      return forecast;
36  })
37  .WithName("GetWeatherForecast");
38
39  app.Run();
40
41  1 reference
42  record WeatherForecast(DateOnly Date, int TemperatureC, string? Summary)
43  {
44      0 references
45      public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);
46  }
```

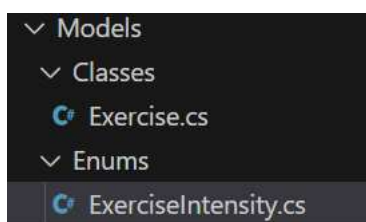
14. Looime kataloogid:

- Models(Seal hoime rakenduses kasutatavate mudelite klasse)
- Data (Seal hoime andmebaasiga suhtlemiseks vajalikud klasse)
- Data/Repos (Seal hoime klasse, mis sisaldavad andmebaasi poole tehtavate päringute meetodeid).
- Controllers(Seal hoime API kontrollereid, mille kaudu kliendid saavad rakenduse poole päringuid teha)
- Looime DataContext klassi



15. Models kataloogis loome kataloogid Klasside ja Enum-ide jaoks ning klass nimega Exercise ja Enum nimega ExerciseIntensity:

- Exercise.cs failis kasutame võtmesõna `class` asemele võtmesõna `record`, sest C# `record` tüübid pakuvad mitmeid eeliseid võrreldes tavalise `class`-iga:
 1. **Immutability:** (`record`-ite väärtuseid ei saa pärast loomist muuta, mis aitab vältida ootamatuid muudatusi)
 2. **Võrdlus väärtuse järgi** (kaks `record`-i loetakse võrdseks, kui nende väljade väärtused on võrdsed)
 3. **Andmete dekompositsioon:** (`record`-id vaikimisi toetavad dekompositsiooni)
- `[JsonConverter(typeof(JsonStringEnumConverter))]` atribuut ütleb, et soovime JSON-is olevad string-id teisendada ümber enum tüübiks ja vastupidi. See võimaldab meil harjutust lisades kasutada string-e „Low“, „Normal“ ja „High“ ja samuti saame vastustes enum-i väärtused string-idena. Ilma selleta peaksime kasutama numbreid.



```

5 using WorkoutApplication.Models.Enums;
6
7 namespace WorkoutApplication.Models.Classes
8 {
9     0 references
10    public record Exercise
11    {
12        0 references
13        public int Id { get; set; }
14        0 references
15        public string? Title { get; set; }
16        0 references
17        public string? Description { get; set; }
18        0 references
19        public ExerciseIntensity Intensity { get; set; }
20        0 references
21        public int RecommendedDurationInSeconds { get; set; }
22        0 references
23        public int RecDurationBefore { get; set; }
24        0 references
25        public int RecDurationAfter { get; set; }
26        0 references
27        public string? RestTimeInstructions { get; set; }
28    }
29 }

```

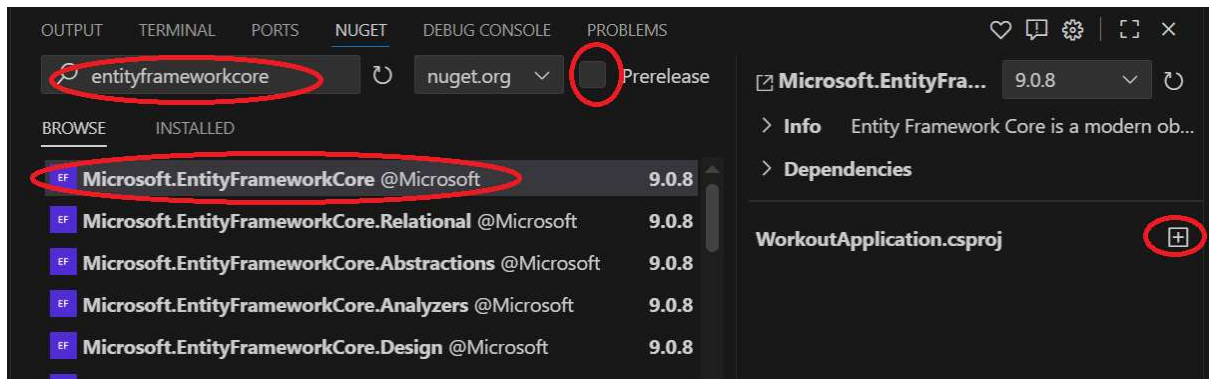
```

using ISA3Demos.Models.Enums;

namespace ISA3Demos.Models;
17 references
public record Exercise {
    10 references
    public int Id { get; init; }
    5 references
    public string? Title { get; init; }
    5 references
    public string? Description { get; init; }
    6 references
    public ExerciseIntensity Intensity { get; init; }
    5 references
    public int RecommendedDurationInSeconds { get; init; }
    5 references
    public int RecDurationBefore { get; init; }
    5 references
    public int RecDurationAfter { get; init; }
    0 references
    public string? RestTimeInstructions { get; init; }
}

```

16. DataContext klassi jaoks on vajalik **EntityFrameworkCore** pakett. Kui võtta prerelease linnuke maha, siis leiab lihtsamalt õige versiooni ülesse.



17. Installime lisaks EntityFrameworkCore.InMemory paketi, mis võimaldab kasutada EntityFramework-is InMemory andmebaasi, mis võimaldab simuleerida reaalse andmebaasi käitumist.

- InMemory andmebaas on andmebaas, mis talletab andmed mälus (RAM-is) ajutiselt, mitte füüsilisel kettal. Seda kasutatakse sageli testimise, prototüüpimise või ajutiste andmete jaoks, kuna andmete salvestamine on väga kiire, kuid need kaovad, kui rakendus sulgub või server taaskäivitub.

18. Projekti .csproj failis nüüd saame näha, et vastavad paketid on projekti installeeritud.

```
WorkoutApplication.csproj
1  <Project Sdk="Microsoft.NET.Sdk.Web">
2
3    <PropertyGroup>
4      <TargetFramework>net9.0</TargetFramework>
5      <Nullable>enable</Nullable>
6      <ImplicitUsings>enable</ImplicitUsings>
7    </PropertyGroup>
8
9    <ItemGroup>
10     <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="9.0.8" />
11     <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.8" />
12     <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="9.0.8" />
13     <PackageReference Include="Scalar.AspNetCore" Version="2.7.2" />
14   </ItemGroup>
15
16 </Project>
17
```

19. DataContext klassi lisame DbSet-i Exercise nimekirja hoidmiseks (ehk näitame, et klassiga Exercise on seotud tabel andmebaasis). Kasutame pärilikkuse ja C# 12 lisatud primary constructor süntaksi.

- Rida `DataContext(DbContextOptions options) : DbContext(options)` loob sama konstruktori kui `public DataContext(DbContextOptions o) : base(o) { }`

```

5 using Microsoft.EntityFrameworkCore;
6 using WorkoutApplication.Models.Classes;
7
8 namespace WorkoutApplication.Data
9 {
10     0 references
11     public class DataContext(DbContextOptions options) : DbContext(options)
12     {
13         0 references
14         public DbSet<Exercise> Exercises { get; set; }
15     }
16 }

```

20. Lisame program.cs faili koodirida, mis registreerib **InMemory** andmebaasi kasutamise teenusekonteineris ASP.NET Core rakenduses. Täpsemalt:

- `builder.Services.AddDbContext<DataContext>` lisab `DataContext`-i rakenduse Dependency Injection konteinerisse, mis võimaldab Inject-ida ja kasutada seda rakenduse erinevates osades
- `options.UseInMemoryDatabase("restDB")` määrab, et andmebaas peaks kasutama InMemory andmebaasi nimega "restDB". See andmebaas hoiab andmeid ainult RAM mälus ja ei salvesta neid püsivalt
- See registreerib DataContext DI teenusena **Scoped** elutsükli määratlusega

Samuti lisame `builder.Services.AddControllers()` ja `app.MapControllers()`; read, mis võimaldavad tuvastada API-kontrollerid ASP.NET Core Web API rakenduses, et oleks võimalik nende poole päringuid teha.

```

1  using Microsoft.EntityFrameworkCore;
2  using Scalar.AspNetCore;
3  using WorkoutApplication.Data;
4
5
6  var builder = WebApplication.CreateBuilder(args);
7
8  // Add services to the container.
9  // Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
10 builder.Services.AddOpenApi();
11 builder.Services.AddControllers();
12
13 builder.Services.AddDbContext<DataContext>(options => options.UseInMemoryDatabase("restDB"));
14
15 var app = builder.Build();
16
17 // Configure the HTTP request pipeline.
18 if (app.Environment.IsDevelopment())
19 {
20     app.MapOpenApi();
21     app.MapScalarApiReference();
22 }
23
24 app.UseHttpsRedirection();
25
26 app.MapControllers();
27
28 app.Run();
29

```

21. Looe Data/Repos katalogis ExercisesRepo klassi ning saame Dependency Injection konteinerist kätte DataContext objekti

```

6  namespace WorkoutApplication.Data.Repos
7  {
8      0 references
9      |   public class ExercisesRepo(DataContext context)
10     |   {
11     |       0 references
12     |       |   private readonly DataContext context = context;
13     |   }
14 }

```

22. Registreerime ExercisesRepo Dependency Injection konteineris **Scoped** elutsükli määratlusega

ASP.NET CORE toetab 3 erinevat põhilisi elutsükli määratlusi:

- **Transient**: Iga kord, kui teenust küsitakse, luuakse uus instants.
- **Scoped**: Üks instants luuakse iga päringu või töötamise konteksti kohta. Näiteks iga HTTP-päringu jaoks luuakse uus instants.
- **Singleton**: Üks instants luuakse rakenduse alguses ja seda kasutatakse kogu rakenduse tööaja jooksul.

```
builder.Services
    .AddDbContext<DataContext>(options => options.UseInMemoryDatabase("restDB"))
    .AddScoped<ExercisesRepo>();
```

23. Looke ExercisesRepo klassis meetodi, mis saab kätte kõik Exercise kirjed andmebaasist

```
5 using Microsoft.EntityFrameworkCore;
6 using WorkoutApplication.Models.Classes;
7
8 namespace WorkoutApplication.Data.Repos
9 {
10     3 references
11     public class ExercisesRepo(DataContext context)
12     {
13         1 reference
14         private readonly DataContext context = context;
15         1 reference
16         public async Task<List<Exercise>> GetAllExercises() => await context.Exercises.ToListAsync();
17     }
18 }
```

24. Looke Controllers kaustas ExercisesController klassi ning saameExerciseRepo objekt kätte kasutades DI(Dependency injection). Looke meetodi, mis saab kätte kõik Exercises andmebaasist ja tagastab neid listina.

```
2 using Microsoft.AspNetCore.Mvc;
3 using WorkoutApplication.Data.Repos;
4
5 namespace WorkoutApplication.Controllers
6 {
7     [ApiController]
8     [Route("api/[controller]")]
9     0 references
10     public class ExercisesController(ExercisesRepo repo) : ControllerBase
11     {
12         1 reference
13         private readonly ExercisesRepo repo = repo;
14         [HttpGet]
15         0 references
16         public async Task<IActionResult> GetAll()
17         {
18             var result = await repo.GetAllExercises();
19             return Ok(result);
20         }
21     }
22 }
23 }
```

25. Testimise eesmärgil oleks hea, kui mõned andmed oleksid juba olemas. Selleks lisame DataContext klassi juurde meetod OnModelCreating soovitud andmetega (ehk teeme Data Seeding):

- **OnModelCreating** meetodit kasutatakse andmebaasi mudeli konfigureerimiseks ja kohandamiseks kasutades Fluent API-d. Fluent API on võimas ja paindlik viis Entity Framework Core konfiguratsiooni täpsustamiseks.

```
6 using WorkoutApplication.Models.Classes;
7 using WorkoutApplication.Models.Enums;
8
9 namespace WorkoutApplication.Data
10 {
11     3 references
12     public class DataContext(DbContextOptions options) : DbContext(options)
13     {
14         1 reference
15         public DbSet<Exercise> Exercises { get; set; }
16
17         0 references
18         protected override void OnModelCreating(ModelBuilder modelBuilder)
19         {
20             base.OnModelCreating(modelBuilder);
21
22             modelBuilder.Entity<Exercise>()
23                 .Property(x => x.Id)
24                 .ValueGeneratedOnAdd();
25
26             modelBuilder.Entity<Exercise>().HasData(
27                 new Exercise
28                 {
29                     Id = 1,
30                     Title = "Running 800m",
31                     Description = "Middle distance",
32                     Intensity = ExerciseIntensity.Normal,
33                     RecommendedDurationInSeconds = 135,
34                     RecDurationAfter = 400,
35                     RecDurationBefore = 100
36                 },
37                 new Exercise
38                 {
39                     Id = 2,
40                     Title = "Running 200m",
41                     Description = "Sprint",
42                     Intensity = ExerciseIntensity.High,
43                     RecommendedDurationInSeconds = 35,
44                     RecDurationAfter = 400,
45                     RecDurationBefore = 100
46                 }
47             );
48         }
49     }
50 }
```

```

44         new Exercise
45         {
46             Id = 3,
47             Title = "Running 400m",
48             Description = "Longer sprint",
49             Intensity = ExerciseIntensity.High,
50             RecommendedDurationInSeconds = 70,
51             RecDurationAfter = 400,
52             RecDurationBefore = 100
53         },
54         new Exercise
55         {
56             Id = 4,
57             Title = "Push-ups",
58             Description = "Makes arms stronger",
59             Intensity = ExerciseIntensity.High,
60             RecommendedDurationInSeconds = 40,
61             RecDurationAfter = 20,
62             RecDurationBefore = 20
63         },
64         new Exercise
65         {
66             Id = 5,
67             Title = "High-jump",
68             Description = "Requires good technique",
69             Intensity = ExerciseIntensity.Normal,
70             RecommendedDurationInSeconds = 30,
71             RecDurationAfter = 30,
72             RecDurationBefore = 10
73         }
74     };
75 }
76 }
77 }

```

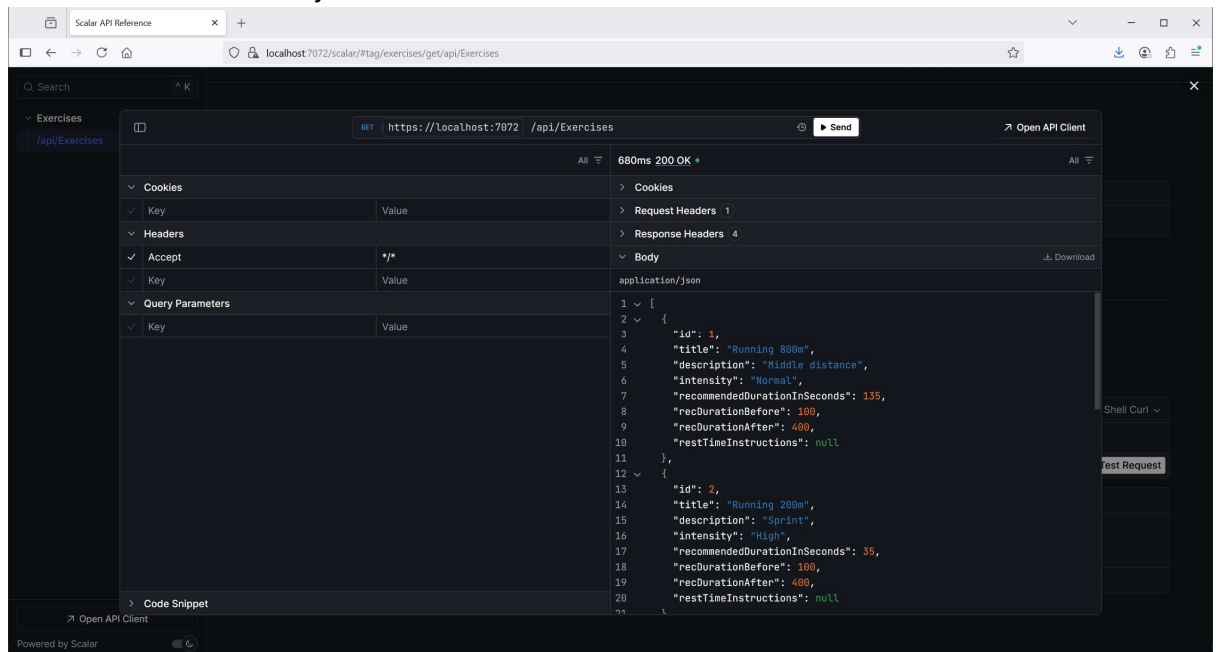
26. Selleks, et rakendada OnModelCreating konfiguratsiooni andmebaasile, lisame program.cs järgmised read (need read loovad Scope-i, saavad Dependency Injection-i kaudu DataContext-i ja loovad andmebaasi vastava konfiguratsiooniga):

```

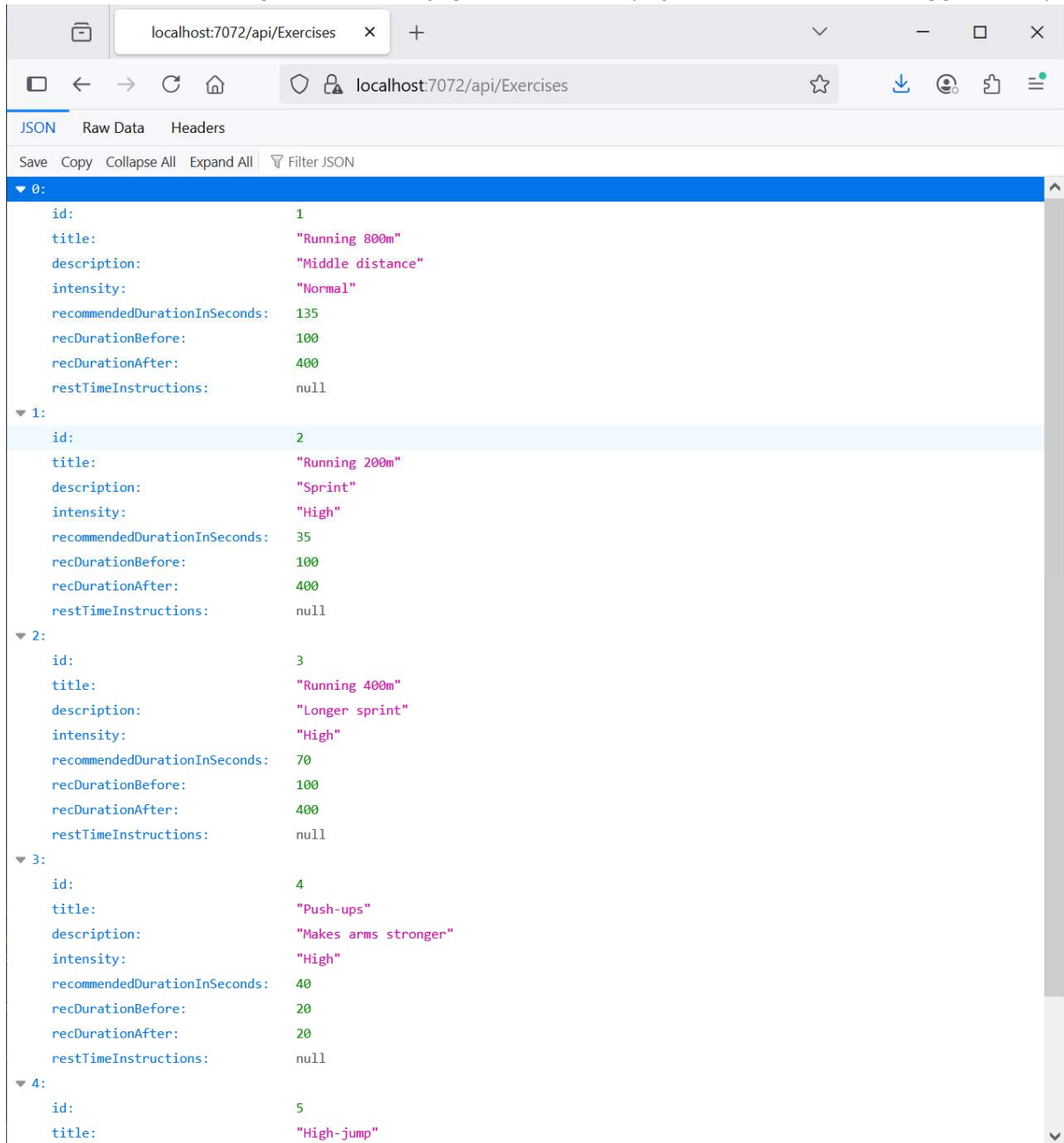
14 builder.Services
15     .AddDbContext<DataContext>(options => options.UseInMemoryDatabase("restDB"))
16     .AddScoped<ExercisesRepo>();
17
18 var app = builder.Build();
19
20 using (var scope = ((IApplicationBuilder)app).ApplicationServices.GetRequiredService<IServiceScopeFactory>().CreateScope())
21 using (var context = scope.ServiceProvider.GetService<DataContext>())
22 {
23     context?.Database.EnsureCreated();
24 }
25
26 // Configure the HTTP request pipeline.
27 if (app.Environment.IsDevelopment())

```

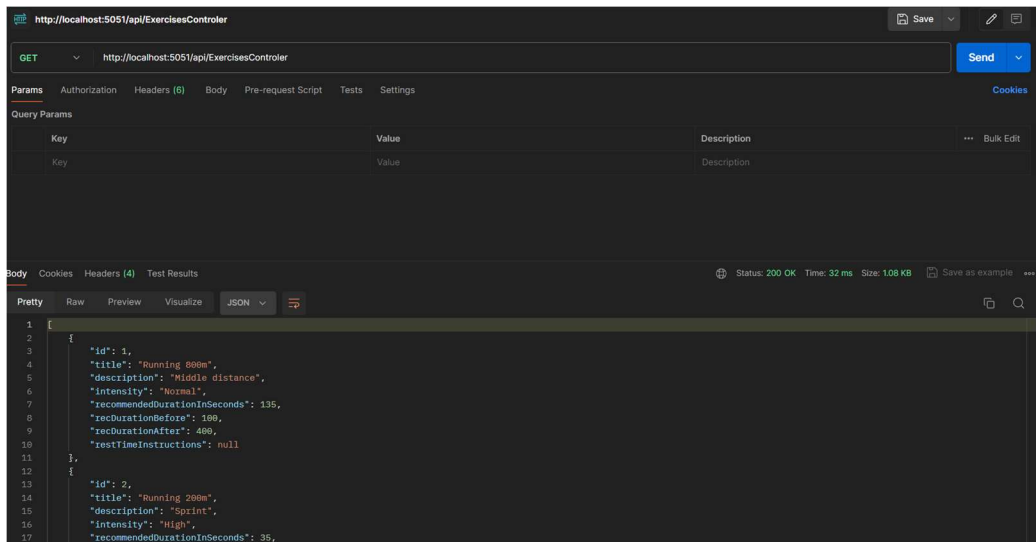
27. Käivitame uuesti programmi. Nüüd peaks saama teha päringu Scalariga ja nägema eelnevalt sisestatud harjutusi:



26. Teeme sama päringu veebilehitsejaga, nt. Chrome (vajalik URL on näha Swaggeri lehel)



28. Teeme sama päringu Postman-iga (Staatus: 200 OK):



29. Lisame Repo klassi ülejäänud meetodid CRUD operatsioonide teostamiseks(Nende meetodite üldistamine ja parem error-handling saab lisada hiljem)

```

using ISA3Demos.Models;
using Microsoft.EntityFrameworkCore;

namespace ISA3Demos.Data.Repos;

3 references
public class ExercisesRepo(DataContext context)
{
    9 references
    private readonly DataContext context = context;

    //CREATE
    0 references
    public async Task<Exercise> SaveExerciseToDb(Exercise exercise) {
        context.Add(exercise);
        await context.SaveChangesAsync();
        return exercise;
    }

    //READ
    1 reference
    public async Task<List<Exercise>> GetAllExercises() => await context.Exercises.ToListAsync();
    1 reference
    public async Task<Exercise?> GetExerciseById(int id) => await context.Exercises.FindAsync(id);
    1 reference
    public async Task<bool> ExerciseExistsInDb(int id) => await context.Exercises.AnyAsync(x => x.Id == id);

    //UPDATE
    0 references
    public async Task<bool> UpdateExercise(int id, Exercise exercise) {

        bool isIdsMatch = id == exercise.Id;
        bool exerciseExists = await ExerciseExistsInDb(id);

        if(!isIdsMatch || !exerciseExists){
            return false;
        }

        context.Update(exercise);
        int updatedRecordsCount = await context.SaveChangesAsync();
        return updatedRecordsCount == 1;
    }

    //DELETE
    0 references
    public async Task<bool> DeleteExerciseById(int id) {
        Exercise? exerciseInDb = await GetExerciseById(id);
        if (exerciseInDb is null){
            return false;
        }
        context.Remove(exerciseInDb);
        int changesCount = await context.SaveChangesAsync();

        return changesCount == 1;
    }
}

```

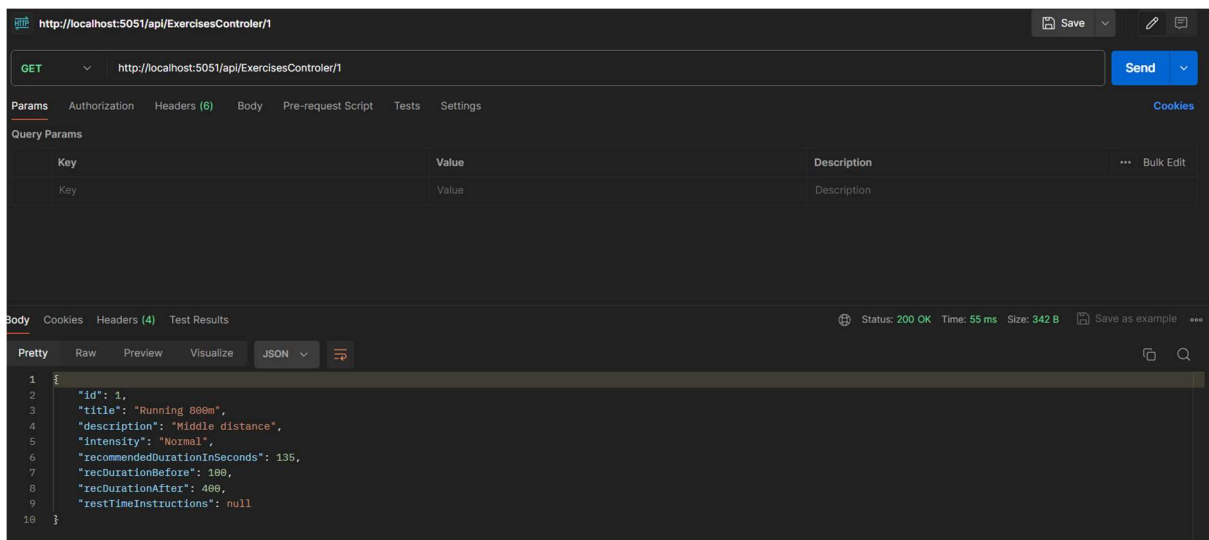
30. Lisame harjutuse küsimiseks id järgi meetodi ExerciseController klassi:

```

[HttpGet("{id}")]
0 references
public async Task<IActionResult> GetExercise(int id){
    var exercise = await repo.GetExerciseById(id);
    if(exercise == null) {
        return NotFound();
    }
    return Ok(exercise);
}

```

31. Testime Postmaniga, lisame soovitud id päringu URLi



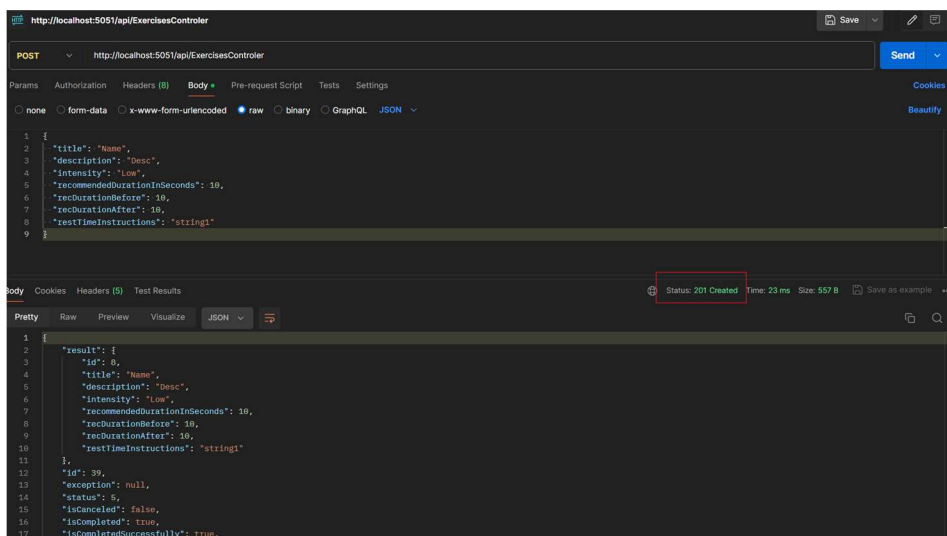
32. Lisame harjutuse loomiseks meetodi ExercisesController klassi

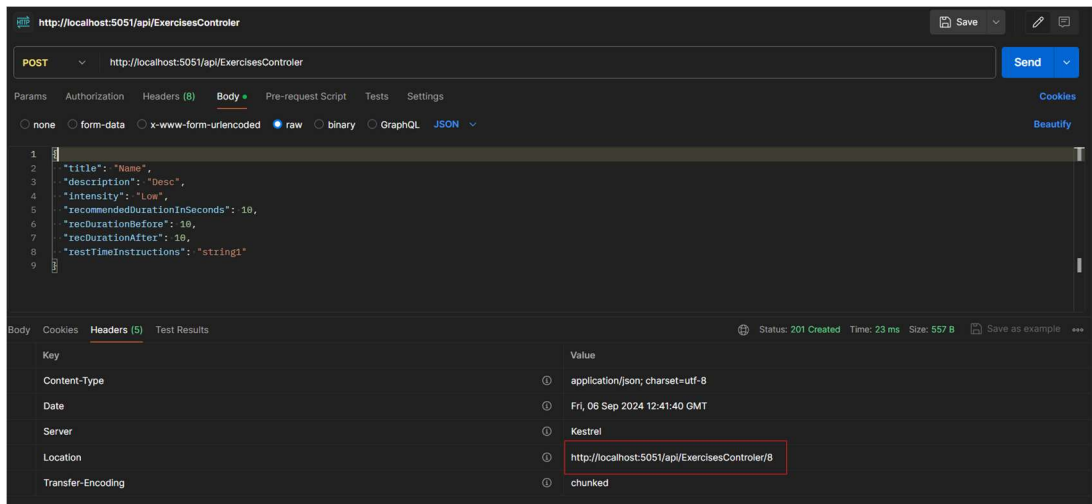
```

[HttpPost]
1 reference
public async Task<IActionResult> SaveExercise([FromBody] Exercise exercise){
    var exerciseExists = await repo.ExerciseExistsInDb(exercise.Id);
    if(exerciseExists) {
        return Conflict();
    }
    var result = repo.SaveExerciseToDb(exercise);
    return CreatedAtAction(nameof(SaveExercise), new { exercise.Id }, result);
}

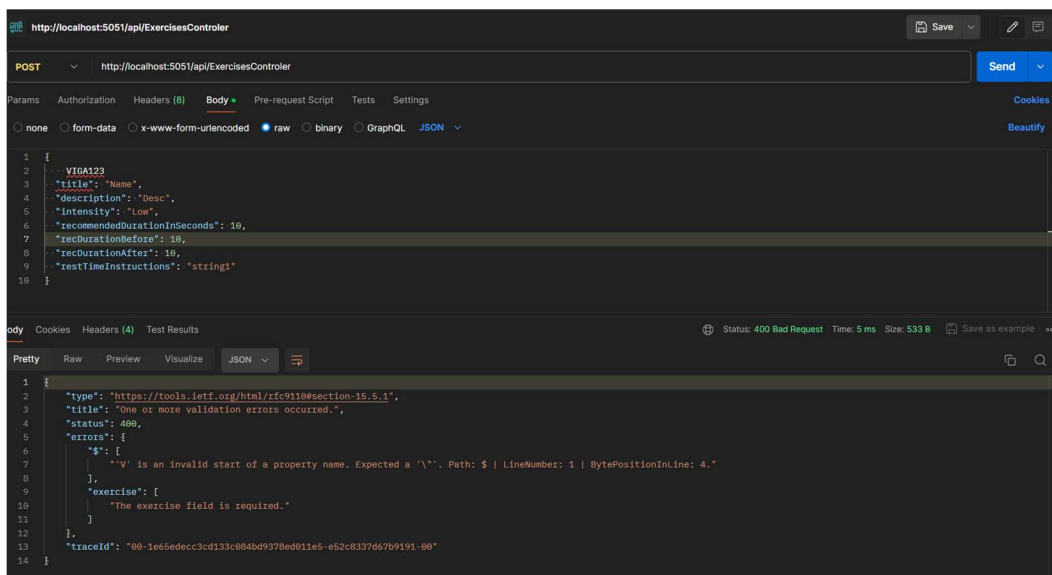
```

33. Harjutuse lisamiseks teeme POST päringu. Vastuseks saame (Staatus 201 - Created):

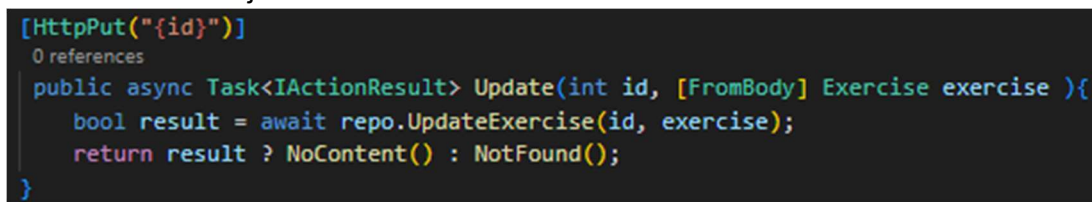




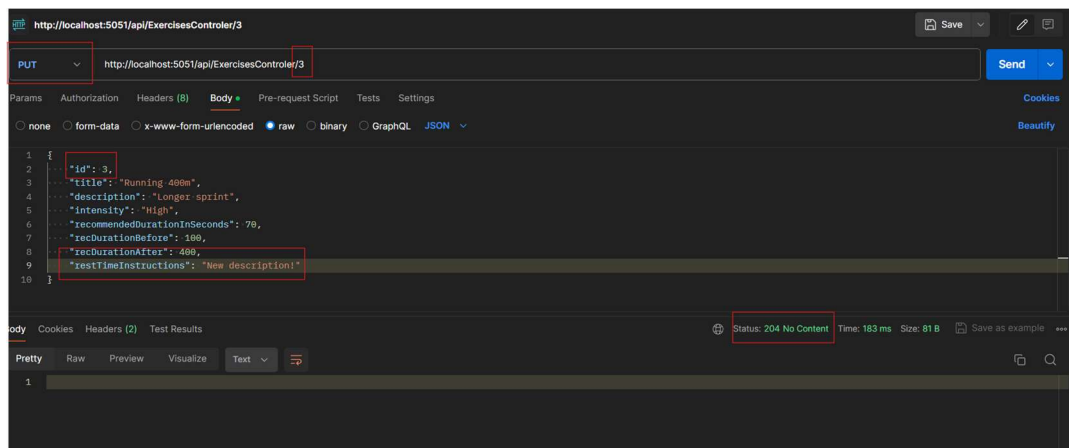
34. Vigase JSON-iga saame vastuse sisuks veateate (Staatus 400 – Bad Request):



35. Lisame harjutuse uuendamiseks meetodi ExercisesController klassi:



36. PUT meetodi abil saab olemasolevaid harjutusi uuendada. Tulemuseks saame (204 - No Content) ja tühja vastuse. See tähendab, et uuendamine õnnestus. Kontrollimiseks saab teha GET päringu



37. Lisame harjutuste nimekirja küsimisele harjutuse intensiivsuse järgi filtreerimise võimaluse (Kasutame IQueryable liides selleks, et dünaamiliselt muuta päring)

```

3 references
public class ExercisesRepo(DataContext context)
{
    9 references
    private readonly DataContext context = context;

    //CREATE
    1 reference
    public async Task<Exercise> SaveExerciseToDb(Exercise exercise) {
        context.Add(exercise);
        await context.SaveChangesAsync();
        return exercise;
    }

    //READ
    1 reference
    public async Task<List<Exercise>> GetAllExercises(ExerciseIntensity? intensity = null) {
        IQueryable<Exercise> query = context.Exercises.AsQueryable();
        if (intensity.HasValue) {
            query = query.Where(x => x.Intensity == intensity);
        }
        return await query.ToListAsync();
    }

    2 references
    public async Task<Exercise?> GetExerciseById(int id) => await context.Exercises.FindAsync(id);
    2 references
    public async Task<bool> ExerciseExistsInDb(int id) => await context.Exercises.AnyAsync(x => x.Id == id);

    //UPDATE
    1 reference
    public async Task<bool> UpdateExercise(int id, Exercise exercise) {

```

```

[ApiController]
[Route("api/[controller]")]
0 references
public class ExercisesController(ExercisesRepo repo) : ControllerBase()
{
    5 references
    private readonly ExercisesRepo repo = repo;

    [HttpGet]
    0 references
    public async Task<IActionResult> GetAll([FromQuery] ExerciseIntensity? intensity){
        var result = await repo.GetAllExercises(intensity);
        return Ok(result);
    }

    [HttpGet("{id}")]
    1 reference
    public async Task<IActionResult> GetExercise(int id){
        var exercise = await repo.GetExerciseById(id);
        if(exercise == null) {
            return NotFound();
        }
        return Ok(exercise);
    }

    [HttpPost]
    0 references
    public async Task<IActionResult> SaveExercise([FromBody] Exercise exercise){
        var exerciseExists = await repo.ExerciseExistsInDb(exercise.Id);
        if(exerciseExists) {
            return Conflict();
        }
        var result = repo.SaveExerciseToDb(exercise);
        return CreatedAtAction(nameof(GetExercise), new { id = exercise.Id }, result);
    }
}

```

38. Filteerimise tulemus Postmanis:

http://localhost:5051/api/ExercisesController?Intensity=Normal

GET http://localhost:5051/api/ExercisesController?Intensity=Normal

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Headers 8 hidden

Key	Value	Description
Key	Value	Description

Body Cookies Headers (4) Test Results

Status: 200 OK Time: 173 ms Size: 541 B Save as example

Pretty Raw Preview Visualize JSON

```

1 {
2   {
3     "id": 1,
4     "title": "Running 800m",
5     "description": "Middle distance",
6     "intensity": "Normal",
7     "recommendedDurationInSeconds": 135,
8     "recDurationBefore": 180,
9     "recDurationAfter": 480,
10    "restTimeInstructions": null
11  },
12  {
13    "id": 5,
14    "title": "High-jump",
15    "description": "Requires good technique",
16    "intensity": "Normal",
17    "recommendedDurationInSeconds": 38,
18    "recDurationBefore": 18,
19    "recDurationAfter": 30,
20    "restTimeInstructions": null
21  }
22 }

```